

La lista enlazada

Insertar sin correr elementos

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Septiembre 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Construir el TAD Lista con nodos enlazados: reservar con `new`, seguir la cadena y liberar con `delete` (OA2, OA3).
- Explicar por qué insertar al frente pasa de $\Theta(n)$ a $\Theta(1)$ y por qué obtener hace el camino contrario (OA1, OA3).
- Reconocer los dos errores que destruyen una lista enlazada: empalmar en el orden equivocado y liberar un nodo todavía enlazado (OA2).
- Ordenar un arreglo con insertion sort, escribir su invariante y acotar su costo en tiempo y en espacio (OA1, OA5).

Contenido I

- 1 Lo que cuesta correr elementos
- 2 Un dato y una dirección
- 3 La lista enlazada
- 4 Agregar sin recorrer
- 5 Lo que cobra cada operación
- 6 Ordenar insertando
- 7 Ejercicios

Contenido I

- 1 Lo que cuesta correr elementos
- 2 Un dato y una dirección
- 3 La lista enlazada
- 4 Agregar sin recorrer
- 5 Lo que cobra cada operación
- 6 Ordenar insertando
- 7 Ejercicios

La lista que ya funciona I

La implementación sobre arreglo guarda los elementos en casillas consecutivas y lleva aparte cuántas están ocupadas.

```
class Lista {  
private:  
    Elemento datos[CAPACIDAD];  
    int n;
```

obtener(2) sobre $\langle 5, 8, 7, 4 \rangle$

5	8	7	4
0	1	2	3

la dirección se calcula: una suma

Pregunta

La lista tiene 100 elementos y se ejecuta `insertar(0, e)`.
¿Cuántas asignaciones a `datos` hace antes de escribir `e`?

Las cien asignaciones I

```
void insertar(int p, Elemento e) {  
    assert(0 <= p && p <= n && n < CAPACIDAD);  
    int i = n;  
    while (i > p) {  
        datos[i] = datos[i - 1];  
        i = i - 1;  
    }  
    datos[p] = e;  
    n = n + 1;  
}
```

Las cien asignaciones II

Con $p = 0$ el ciclo va de $i = n$ hasta $i = 1$: hace n asignaciones.
Con 100 elementos, 100; con un millón, un millón.

`insertar(0, 2) sobre  $\langle 5, 8, 7 \rangle$`

antes	5	8	7	
después	2	5	8	7

tres elementos se movieron

Y hay un segundo problema

CAPACIDAD se fija al compilar. La lista número 101 no cabe, y el `assert` lo detiene.

Contenido I

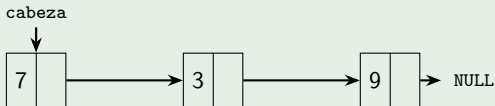
- 1 Lo que cuesta correr elementos
- 2 Un dato y una dirección**
- 3 La lista enlazada
- 4 Agregar sin recorrer
- 5 Lo que cobra cada operación
- 6 Ordenar insertando
- 7 Ejercicios

El nodo I

Si los elementos no van en casillas consecutivas, cada uno tiene que decir dónde está el siguiente. Eso es un nodo: el dato y una dirección.

```
struct Nodo {  
    Elemento dato;  
    Nodo *siguiente;  
};
```

Tres nodos enlazados



El nodo ocupa lo que ocupa el dato más lo que ocupa un puntero. La lista de n elementos ocupa $\Theta(n)$, igual que el arreglo, pero paga un puntero por elemento y no reserva casillas de sobra.

new y delete I

Reservar en tiempo de ejecución ya se hizo con `malloc` y `free`. En C++ el par es `new` y `delete`: `new` pide la memoria y devuelve un puntero del tipo pedido, sin el molde que exigía `malloc`.

C	C++
<code>(Nodo*) malloc(sizeof(Nodo))</code>	<code>new Nodo</code>
<code>free(p)</code>	<code>delete p</code>

La regla no cambió

Cada `new` lleva su `delete`. Lo que se reserva y no se libera queda ocupado hasta que el programa termina.

Tres nodos a mano l

```
Nodo *a = new Nodo;  
Nodo *b = new Nodo;  
Nodo *c = new Nodo;  
  
a->dato = 7;  
b->dato = 3;  
c->dato = 9;  
  
a->siguiente = b;  
b->siguiente = c;  
c->siguiente = NULL;    // el ultimo no apunta a nada
```

Tres nodos a mano II

El recorrido no usa índices. Se arranca en el primero y se sigue la cadena hasta NULL.

```
Nodo *actual = a;
while (actual != NULL) {
    std::cout << actual->dato << " ";
    actual = actual->siguiente;
}
```

Tres nodos a mano III

Al liberar se guarda el siguiente *antes* de borrar: después de delete el nodo ya no se puede leer.

```
actual = a;
while (actual != NULL) {
    Nodo *muerto = actual;
    actual = actual->siguiente;
    delete muerto;
}
```

Salida

7 3 9

Contenido I

- 1 Lo que cuesta correr elementos
- 2 Un dato y una dirección
- 3 La lista enlazada**
- 4 Agregar sin recorrer
- 5 Lo que cobra cada operación
- 6 Ordenar insertando
- 7 Ejercicios

El estado de la lista I

La clase guarda dos cosas: dónde empieza la cadena y cuántos elementos tiene.

```
class Lista {  
private:  
    Nodo *cabeza;  
    int n;  
public:  
    Lista() {  
        cabeza = NULL;  
        n = 0;  
    }  
}
```

El estado de la lista II

Lo que toda operación debe dejar cierto

Sea $v_0, v_1, \dots, v_{n-1}$ la sucesión de nodos que se alcanza desde cabeza siguiendo el campo siguiente.

$$I_1 : \text{cabeza} = \text{NULL} \iff n = 0$$

$$I_2 : \forall i, 0 \leq i < n - 1 : \text{el campo siguiente de } v_i \text{ es } v_{i+1}$$

$$I_3 : \text{el campo siguiente de } v_{n-1} \text{ es NULL}$$

Una operación que rompa I_2 deja elementos inalcanzables; una que rompa I_3 deja el recorrido leyendo memoria que no es suya.

Llegar a la posición p I

En el arreglo la posición se calculaba. Aquí se camina: para llegar a v_p hay que pasar por $v_0, \dots, v_{p-1}$.

```
// Devuelve el nodo de la posición p. Exige  $0 \leq p < n$ .
Nodo *nodoEn(int p) {
    Nodo *actual = cabeza;
    int i = 0;
    while (i < p) {
        actual = actual->siguiente;
        i = i + 1;
    }
    return actual;
}
```

```
Elemento obtener(int p) {
    assert(0 <= p && p < n);
    return nodoEn(p)->dato;
}
```

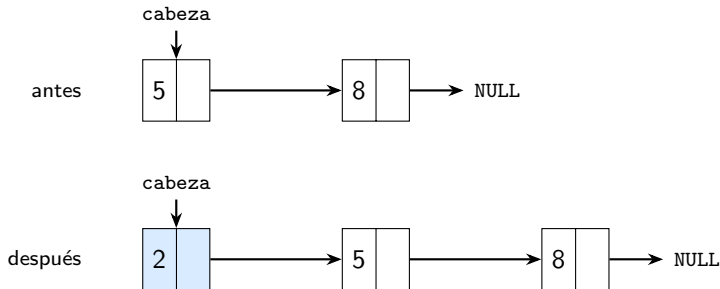
Llegar a la posición p II

`nodoEn(p)` da p pasos: es $\Theta(p)$, y en el peor caso $\Theta(n)$. Lo que el arreglo hacía en $O(1)$ aquí se paga recorriendo.

Insertar al frente I

Con $p = 0$ no hay nada que correr: el nodo nuevo apunta a la cabeza vieja y la cabeza pasa a ser el nodo nuevo.

```
nuevo->siguiente = cabeza;  
cabeza = nuevo;
```



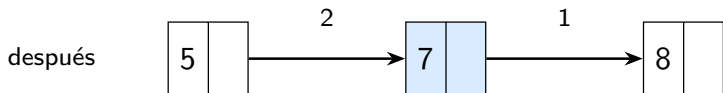
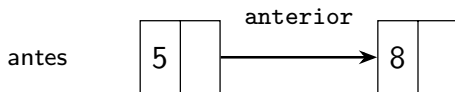
Dos escrituras de puntero, sin importar cuántos elementos haya: $\Theta(1)$.

Insertar en medio I

Para insertar en la posición $p > 0$ hay que llegar al nodo *anterior*, porque es el que tiene que apuntar al nuevo.

```
void insertar(int p, Elemento e) {
    assert(0 <= p && p <= n);
    Nodo *nuevo = new Nodo;
    nuevo->dato = e;
    if (p == 0) {
        nuevo->siguiente = cabeza;
        cabeza = nuevo;
    } else {
        Nodo *anterior = nodoEn(p - 1);
        nuevo->siguiente = anterior->siguiente;
        anterior->siguiente = nuevo;
    }
    n = n + 1;
}
```

Insertar en medio II



El orden de las dos líneas no es libre

Primero el nodo nuevo toma el resto de la cadena (1) y después el anterior lo toma a él (2). Al revés, anterior->siguiente ya vale nuevo cuando se lee, y la cadena a partir de ese punto queda inalcanzable: se pierde la cola de la lista.

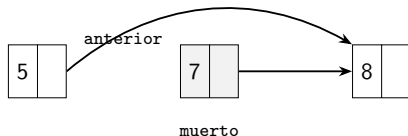
Insertar en medio III

El empalme es $\Theta(1)$; llegar al anterior es $\Theta(p)$. El total es $\Theta(p)$, y $\Theta(1)$ cuando $p = 0$.

Eliminar l

```
void eliminar(int p) {  
    assert(0 <= p && p < n);  
    Nodo *muerto;  
    if (p == 0) {  
        muerto = cabeza;  
        cabeza = cabeza->siguiente;  
    } else {  
        Nodo *anterior = nodoEn(p - 1);  
        muerto = anterior->siguiente;  
        anterior->siguiente = muerto->siguiente;  
    }  
    delete muerto;  
    n = n - 1;  
}
```

Eliminar II



el nodo queda fuera de la cadena
y solo entonces se libera

Los dos errores que destruyen la lista

- delete antes de desenlazar: anterior queda apuntando a memoria liberada, un puntero colgante. El siguiente recorrido lee lo que ya no es suyo.
- Desenlazar sin delete: el nodo queda reservado y sin dueño hasta que el programa termina. Es la fuga.

Liberar la lista entera I

El arreglo se iba solo cuando la lista salía de alcance. Los nodos no: cada uno se pidió con `new` y hay que devolverlo.

```
~Lista() {  
    while (cabeza != NULL) {  
        Nodo *muerto = cabeza;  
        cabeza = cabeza->siguiente;  
        delete muerto;  
    }  
}
```

Se guarda la cabeza en `muerto`, se avanza y solo entonces se libera. Si se liberara primero, `cabeza->siguiente` leería un nodo muerto y el resto de la lista se perdería.

El destructor recorre una vez: $\Theta(n)$ en tiempo y $\Theta(1)$ aparte de la lista.

El mismo programa, la otra implementación I

El código que usa la lista no cambió ni una línea: el contrato es el mismo y lo que cambió está debajo.

```
Lista l;  
l.agregar(5);  
l.agregar(8);  
imprimir(l);           // 5 8  
l.insertar(1, 7);  
imprimir(l);           // 5 7 8  
l.insertar(0, 2);  
imprimir(l);           // 2 5 7 8  
l.eliminar(2);  
imprimir(l);           // 2 5 8
```

El mismo programa, la otra implementación II

Salida

5 8

5 7 8

2 5 7 8

2 5 8

Contenido I

- 1 Lo que cuesta correr elementos
- 2 Un dato y una dirección
- 3 La lista enlazada
- 4 Agregar sin recorrer**
- 5 Lo que cobra cada operación
- 6 Ordenar insertando
- 7 Ejercicios

Agregar al final cuesta toda la lista I

agregar es insertar en la posición n , y para eso hay que llegar al último nodo.

```
void agregar(Elemento e) {  
    insertar(n, e);  
}
```

insertar(n , e) llama a `nodoEn( $n - 1$ )`: recorre la lista entera. Agregar es $\Theta(n)$, justo la operación que en el arreglo costaba $\Theta(1)$.

Pregunta

¿Qué haría falta guardar para que agregar no recorra nada?

Un puntero al último l

Si la clase recuerda dónde termina la cadena, agregar escribe dos punteros y ya.

```
Nodo *cabeza;  
Nodo *ultimo;  
int n;
```

Un puntero al último II

```
void agregar(Elemento e) {  
    Nodo *nuevo = new Nodo;  
    nuevo->dato = e;  
    nuevo->siguiente = NULL;  
    if (cabeza == NULL) {  
        cabeza = nuevo;  
    } else {  
        ultimo->siguiente = nuevo;  
    }  
    ultimo = nuevo;  
    n = n + 1;  
}
```

Un puntero al último III

El dato nuevo hay que mantenerlo: aparece un invariante más y eliminar gana dos casos.

Lo que se agrega a l_1 , l_2 , l_3

l_4 : si $n > 0$, ultimo = v_{n-1} ; si $n = 0$, ultimo = NULL.

Un puntero al último IV

```
// dentro de eliminar
if (p == 0) {
    muerto = cabeza;
    cabeza = cabeza->siguiente;
    if (cabeza == NULL) {
        ultimo = NULL;           // la lista quedo vacia
    }
} else {
    Nodo *anterior = nodoEn(p - 1);
    muerto = anterior->siguiente;
    anterior->siguiente = muerto->siguiente;
    if (muerto == ultimo) {
        ultimo = anterior;       // se borro el ultimo
    }
}
```

Un puntero al último V

Las dos versiones quedan: la primera es más corta y agrega en $\Theta(n)$; la segunda agrega en $\Theta(1)$ y cobra el precio en los dos casos nuevos de eliminar, que son justamente donde aparecen los errores.

Contenido I

- 1 Lo que cuesta correr elementos
- 2 Un dato y una dirección
- 3 La lista enlazada
- 4 Agregar sin recorrer
- 5 Lo que cobra cada operación**
- 6 Ordenar insertando
- 7 Ejercicios

Las dos implementaciones del mismo contrato I

Operación	Estática	Enlazada	Por qué
obtener(p), asignar(p, e)	$\Theta(1)$	$\Theta(p)$	se calcula la dirección / se camina hasta el nodo
insertar(0, e)	$\Theta(n)$	$\Theta(1)$	se corre todo / se escriben dos punteros
insertar(p, e)	$\Theta(n - p)$	$\Theta(p)$	se corre lo que sigue / se camina hasta el anterior
eliminar(p)	$\Theta(n - p)$	$\Theta(p)$	se corre lo que sigue / se camina hasta el anterior
agregar(e)	$\Theta(1)$	$\Theta(n)$	la casilla n ya está / hay que llegar al último
agregar(e) con ultimo	$\Theta(1)$	$\Theta(1)$	se guarda dónde termina la cadena
tamano(), vacia()	$\Theta(1)$	$\Theta(1)$	el contador está guardado
liberar la lista	$\Theta(1)$	$\Theta(n)$	el arreglo se va solo / un delete por nodo
espacio de la estructura	$\Theta(\text{CAPACIDAD})$	$\Theta(n)$	casillas reservadas de antemano / un nodo por elemento
espacio aparte por operación	$\Theta(1)$	$\Theta(1)$	punteros sueltos

Ninguna gana en todo. El arreglo cobra al modificar y regala el acceso; la enlazada cobra el acceso y regala la modificación en el frente. La estructura se escoge por las operaciones que el programa hace más veces.

La diferencia, medida I

Insertar al frente n veces, contando lo que mueve cada una.

```
// arreglo: cada insercion corre todo lo que ya hay  
int i = usados;  
while (i > 0) {  
    datos[i] = datos[i - 1];  
    i = i - 1;  
    movidas = movidas + 1;  
}
```

```
// enlazada: dos escrituras, sin importar el tamaño  
nuevo->siguiente = cabeza;  
cabeza = nuevo;  
escrituras = escrituras + 2;
```

La diferencia, medida II

Salida

n	arreglo	enlazada
10	45	20
100	4 950	200
1000	499 500	2 000

El arreglo hace $0 + 1 + \cdots + (n - 1) = \frac{n(n-1)}{2}$ movidas: con $n = 1000$ son 499 500. La enlazada hace $2n$. Multiplicar n por 10 multiplica por 100 el trabajo de una y por 10 el de la otra.

Contenido I

- 1 Lo que cuesta correr elementos
- 2 Un dato y una dirección
- 3 La lista enlazada
- 4 Agregar sin recorrer
- 5 Lo que cobra cada operación
- 6 Ordenar insertando**
- 7 Ejercicios

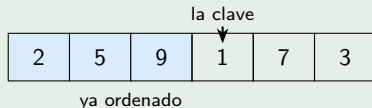
La mano de cartas I

Quien reparte cartas las va acomodando: toma la que llega y la mete en su lugar entre las que ya tiene ordenadas, corriendo las mayores una posición a la derecha.

Pregunta

En $\langle 5, 2, 9, 1, 7, 3 \rangle$, al llegar al 1 hay tres elementos ya ordenados a su izquierda: $\langle 2, 5, 9 \rangle$. ¿Cuántos se corren para meter el 1?

Los tres



El 9, el 5 y el 2 son mayores que 1, así que los tres se corren y el 1 queda al principio.

El algoritmo I

```
// Ordena a[0..n) y devuelve cuantas veces corrio un elemento.
long insertionSort(Elemento a[], int n) {
    long corrimientos = 0;
    int j = 1;
    while (j < n) {
        Elemento clave = a[j];
        int i = j - 1;
        while (i >= 0 && a[i] > clave) {
            a[i + 1] = a[i];
            i = i - 1;
            corrimientos = corrimientos + 1;
        }
        a[i + 1] = clave;
        j = j + 1;
    }
    return corrimientos;
}
```

El algoritmo II

La clave se guarda aparte porque su casilla se va a sobrescribir con el primer corrimiento. El ciclo interno para cuando encuentra un elemento que no es mayor, y ahí mismo está el hueco donde va.

El invariante I

Al empezar cada vuelta del ciclo externo, con índice j

Sea A el arreglo en ese momento y E el arreglo de entrada.

$$I_1 : \forall a, b, 0 \leq a < b < j : A[a] \leq A[b]$$

$$I_2 : \{\{A[0], \dots, A[j-1]\}\} = \{\{E[0], \dots, E[j-1]\}\}$$

$$I_3 : \forall i, j \leq i < n : A[i] = E[i]$$

I_1 dice que el trozo $A[0..j)$ está en orden; I_2 , que son los mismos valores de la entrada y no otros, contados con repetición; I_3 , que lo que falta no se ha tocado.

Cuando el ciclo termina, $j = n$, y I_1 con I_2 evaluados en $j = n$ dicen exactamente que el arreglo completo está ordenado y contiene los mismos valores de la entrada.

El invariante II

El ciclo interno sostiene I_1 para la vuelta siguiente: corre a la derecha todos los mayores que la clave y deja el hueco en $i + 1$, que es la primera posición donde la clave no rompe el orden.

La traza I

Sobre $\langle 5, 2, 9, 1, 7, 3 \rangle$:

j	clave	corrimientos	el arreglo al terminar la vuelta
1	2	1	$\langle 2, 5, 9, 1, 7, 3 \rangle$
2	9	0	$\langle 2, 5, 9, 1, 7, 3 \rangle$
3	1	3	$\langle 1, 2, 5, 9, 7, 3 \rangle$
4	7	1	$\langle 1, 2, 5, 7, 9, 3 \rangle$
5	3	3	$\langle 1, 2, 3, 5, 7, 9 \rangle$
total		8	

En $j = 2$ la clave es 9 y el ciclo interno no entra: 9 ya es mayor que todo lo que tiene a la izquierda. Esa vuelta cuesta una comparación y nada más.

Lo que cuesta I

Caso	Tiempo	Por qué
entrada ya ordenada	$\Theta(n)$	el ciclo interno nunca entra: una comparación por vuelta
entrada al revés	$\Theta(n^2)$	la vuelta j corre los j elementos de su izquierda
cualquier entrada	$O(n^2)$, $\Omega(n)$	entre esos dos extremos
espacio aparte	$\Theta(1)$	clave, i y j ; se ordena en el sitio

En el peor caso los corrimientos suman

$$1 + 2 + \cdots + (n - 1) = \frac{n(n-1)}{2}.$$

Lo que cuesta II

Medido con $n = 6$

mezclado $\langle 5, 2, 9, 1, 7, 3 \rangle$	8 corrimientos
ya ordenado $\langle 1, 2, 3, 5, 7, 9 \rangle$	0 corrimientos
al revés $\langle 9, 7, 5, 3, 2, 1 \rangle$	15 corrimientos

$$\text{Y } \frac{6 \cdot 5}{2} = 15.$$

Insertion sort sobre la enlazada I

En la lista enlazada el corrimiento desaparece: meter un nodo en su lugar es un empalme, $\Theta(1)$, y no se mueve ningún dato.

Y sin embargo

Encontrar dónde va exige caminar desde la cabeza. La vuelta j puede recorrer j nodos, así que el peor caso sigue siendo $\Theta(n^2)$: se cambió mover datos por seguir punteros, y son la misma cantidad de pasos.

Lo que la enlazada no deja hacer

El ciclo interno de insertion sort va hacia atrás, de $a[i]$ a $a[i - 1]$. Un nodo sabe cuál es el siguiente y no cuál es el anterior, así que ese recorrido no existe: hay que buscar desde el principio.

Bajar de $\Theta(n^2)$ pide otra idea, y no es acomodar un elemento a la vez.

Contenido I

- 1 Lo que cuesta correr elementos
- 2 Un dato y una dirección
- 3 La lista enlazada
- 4 Agregar sin recorrer
- 5 Lo que cobra cada operación
- 6 Ordenar insertando
- 7 Ejercicios**

Ejercicio en parejas: contar los mayores I

Escriba una función que reciba una Lista enlazada y un Elemento x, y devuelva cuántos elementos de la lista son mayores que x.

```
int contarMayores(Lista &l, Elemento x) {  
    int cuantos = 0;  
    int i = 0;  
    while (i < l.tamano()) {  
        if (l.obtener(i) > x) {  
            cuantos = cuantos + 1;  
        }  
        i = i + 1;  
    }  
    return cuantos;  
}
```

Pregunta

Esta versión es correcta. ¿Cuánto cuesta sobre la lista enlazada?

Ejercicio en parejas: contar los mayores II

La respuesta

obtener(i) camina i pasos. El ciclo llama a obtener con $i = 0, 1, \dots, n-1$, así que da $0 + 1 + \dots + (n-1) = \frac{n(n-1)}{2}$ pasos en total: $\Theta(n^2)$.

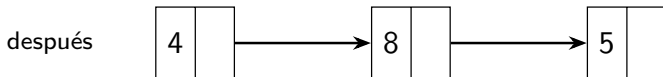
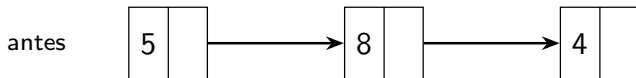
El mismo código sobre la lista estática es $\Theta(n)$, porque allí obtener es $\Theta(1)$. Un recorrido que no cambia de forma cambia de orden de crecimiento al cambiar la implementación de debajo.

La segunda parte

Con acceso al campo cabeza, escriba la versión que recorre con un puntero y hace un solo paso por nodo. ¿Cuál es su costo?

Ejercicio en parejas: dar vuelta la lista I

Invierta la lista enlazada dando vuelta a los punteros, sin reservar nodos nuevos y sin mover datos. Al terminar, la cabeza debe ser el que era último.



Ejercicio en parejas: dar vuelta la lista II

La pista

Se necesitan tres punteros a la vez: el nodo actual, el que ya quedó dado vuelta detrás de él y el siguiente, que hay que guardar antes de reescribir `actual->siguiente`.

Una sola pasada: $\Theta(n)$ en tiempo y $\Theta(1)$ aparte de la lista.
Compárelo con darla vuelta usando `insertar(0, e)` sobre una lista nueva.

Propuestos para la casa I

- 1 Buscar en la enlazada.** Escriba `int buscar(Lista &l, Elemento x)` que devuelva la posición de la primera aparición de `x`, o `-1`. ¿Cuánto cuesta en el mejor y en el peor caso?
- 2 El último sin contador.** Si la clase no guardara `n`, ¿cómo se obtiene `tamano()`? ¿En cuánto pasa a costar?
- 3 La lista de dos puntas.** Agregue a cada nodo un campo anterior. ¿Qué operaciones bajan de costo y qué tiene que mantener insertar que antes no mantenía?
- 4 Ordenada al insertar.** Escriba `insertarOrdenado(Lista &l, Elemento e)` que deje la lista en orden si ya lo estaba. ¿Qué cuesta ordenar n elementos llamándola n veces?
- 5 Los corrimientos exactos.** Para $\langle 4, 3, 2, 1 \rangle$, ¿cuántos corrimientos hace insertion sort? Compruébelo con la fórmula $\frac{n(n-1)}{2}$.

Resumen I

- Un nodo guarda el dato y la dirección del siguiente. La lista es la cadena que se alcanza desde cabeza y termina en NULL. Cada new lleva su delete, y el destructor recorre liberando.
- Al empalmar, el nodo nuevo toma el resto de la cadena antes de que el anterior lo tome a él; al eliminar, se desenlaza antes de liberar. Invertir cualquiera de los dos órdenes pierde la cola o deja un puntero colgante.
- insertar al frente pasa de $\Theta(n)$ a $\Theta(1)$; obtener pasa de $\Theta(1)$ a $\Theta(p)$. Guardar un puntero al último baja agregar a $\Theta(1)$ y agrega dos casos a eliminar.
- Insertion sort ordena en el sitio con $\Theta(1)$ de espacio: $\Theta(n)$ si la entrada ya está ordenada y $\Theta(n^2)$ si está al revés, con $\frac{n(n-1)}{2}$ corrimientos.

Resumen II

Lo que sigue

Ordenar partiendo la lista en dos, ordenando cada mitad y mezclando las dos mitades ordenadas: merge sort, y por qué eso baja el peor caso de $\Theta(n^2)$ a $\Theta(n \log n)$.

Referencias I

-  R. Thareja. *Data Structures Using C*. Oxford University Press, 2018. Capítulo 6, listas enlazadas.
-  T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.^a ed. MIT Press, 2022. Sección 2.1, insertion sort; sección 10.2, listas enlazadas.
-  N. Kalicharan. *Data Structures in C*. 2008. Listas enlazadas y sus operaciones.